Übungsblatt 6: Server-Verfahren für ereignisgetriebene Last

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: CPU-Budget und periodische Interferenz (8 Punkte)

Alle Werte sind Lehrannahmen in Millisekunden. Ein Prozessor, feste Prioritäten, vollständige Präemption, keine Sperren, Selbstsuspendierung oder Zusatzkosten. Periodische Jobs werden synchron bei null freigegeben, ihre Deadline entspricht der Periode. Der Polling Server wird für diesen Nachweis konservativ bei jeder Freigabe mit seinem vollen Budget berücksichtigt.

Pfad	CPU-Zeit / Budget	Periode	Priorität
Regelung	2	7	höchste
Polling Server	2	14	mittlere
Zustandsmeldung	3	28	niedrigste

Verwenden Sie $R_i^{(0)} = C_i$ und

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j.$$

Für den Server gilt in dieser Rechnung $C_S = Q_S$. $hp(i)$ enthält die höher priorisierten Pfade.

1. In einem getrennten Lauf rechnet der Server von 1 bis 2 und von 5 bis 6 ms. Welches Budget verbraucht er, welches Intervall liegt zwischen erstem Start und Ende und wie viel bleibt von 2 ms Budget? (2 Punkte)
2. Berechnen Sie die Gesamtauslastung des Tabellenmodells. (2 Punkte)
3. Bestimmen Sie die Antwortzeitfixpunkte für Server und Zustandsmeldung. (3 Punkte)
4. Bewerten Sie deren periodische Modellfristen. Ist die Serverantwort automatisch die Antwort jeder bei ihm eingereihten Anfrage? (1 Punkt)

Musterlösung:

1. Eigene CPU-Zeit: $(2-1)+(6-5)=2$ ms. Vom ersten Start bis Ende vergehen 5 ms; die Unterbrechung verbraucht kein eigenes Serverbudget. Restbudget null. Eine Antwortzeit ab Anfrageankunft ist ohne deren Zeitpunkt nicht berechenbar.
2. $U = 2/7 + 2/14 + 3/28 = 15/28 \approx 0{,}5357$, also rund 53,57 %. Der Serveranteil beträgt $2/14 = 1/7 \approx 14{,}29\,\%$.
3. Server: $2 \to 2 + \lceil 2/7 \rceil 2 = 4 \to 4$ ms. Zustandsmeldung: $3 \to 3 + \lceil 3/7 \rceil 2 + \lceil 3/14 \rceil 2 = 7 \to 7$ ms. Bei $R = 7$ liegt genau eine höher priorisierte 7-ms-Freigabe im halboffenen Antwortintervall, deshalb bleibt der Fixpunkt sieben.

4. $R_S = 4 \leq 14$ und $R_Z = 7 \leq 28$: beide Modellfristen erfüllt. Eine einzelne Anfrage kann vor der nächsten Pollgelegenheit und hinter älteren Anfragen warten. Ihr Ankunfts-bis-Abschluss-Intervall ist deshalb eine andere Größe. Die Rechnung gilt hier für den Polling Server und wird nicht unverändert auf einen Deferrable Server übertragen.

Aufgabe 2: Ein Polling Server mit unterbrochener Anfrage (8 Punkte)

Ein Server hat $Q_S = 3$ ms und $T_S = 12$ ms. Keine andere Arbeit, keine Zusatzkosten, FIFO-Reihenfolge. Die logischen Pollzeitpunkte sind 0, 12, 24, … ms. Neue Ankünfte am Pollzeitpunkt zählen vor der Pollprüfung. Bei leerem Poll verfällt das gesamte Budget. Wird die Arbeit später vollständig leer, verfällt auch Restbudget. Eine angefangene Anfrage darf bei Budgetende unterbrochen werden; ihr Rest wird vor jüngeren Anfragen fortgesetzt. Es gibt genug Pufferplatz.

Anfrage	Ankunft	eigene CPU-Zeit
A	2	1
B	5	1
C	11	2
D	13	1

1. Zeichnen Sie Serverausführung und Leerlauf für $[0, 27)$ ms. (2 Punkte)
2. Bestimmen Sie Abschluss und Antwortzeit jeder Anfrage. (3 Punkte)
3. Wie viel Restbudget bleibt nach dem letzten Abschluss und was geschieht damit? Bestimmen Sie außerdem das kleinste ganzzahlige Budget Q_S in ms, mit dem D bei unverändertem $T_S = 12$ ms spätestens bei 20 ms fertig wird. Begründen Sie auch, warum ein kleineres Budget nicht genügt. (3 Punkte)

Musterlösung: Bei null ist die Queue leer; das Budget verfällt. Die Ankünfte bei 2, 5 und 11 ms eröffnen keine neue Bediengelegenheit. Bei 12 werden drei CPU-Millisekunden verfügbar.

Intervall in ms	Arbeit
$[0, 12)$	Leerlauf
$[12, 13)$	A
$[13, 14)$	B; D ist inzwischen hinter C eingereiht
$[14, 15)$	erster Teil von C
$[15, 24)$	Leerlauf, obwohl C und D noch Arbeit haben
$[24, 25)$	Rest von C
$[25, 26)$	D
$[26, 27)$	Leerlauf

Anfrage	Abschluss	Antwort ab Ankunft
A	13	$13 - 2 = 11$ ms
B	14	$14 - 5 = 9$ ms
C	25	$25 - 11 = 14$ ms
D	26	$26 - 13 = 13$ ms

Bei 26 ms bleibt eine CPU-Millisekunde übrig. Sie verfällt, weil keine Anfrage mehr offen ist. Der Server beginnt eine neue volle Periode nicht schon beim Eintreffen von D. Der lange Leerlauf von 15 bis 24 ist eine gewollte Budgetpause, kein fehlgeschlagener Scheduler.

Für den Abschluss von D bis 20 ms muss das Budget der Gelegenheit bei 12 ms für A, B, C und D reichen: $Q_S \geq 1+1+2+1 = 5$ ms. Bei $Q_S = 5$ laufen A von 12 bis 13, B von 13 bis 14, C von 14 bis 16 und D von 16 bis 17 ms. D ist rechtzeitig fertig. Mit höchstens vier Einheiten bleibt für D kein Budget; die nächste Gelegenheit bei 24 ms liegt schon hinter der Frist. Das kleinste ganzzahlige Budget ist daher fünf. Diese Dimensionierung gilt unter der gegebenen Annahme, dass keine andere Arbeit die Ausführung verdrängt.

Aufgabe 3: Deferrable und Sporadic Server vergleichen (8 Punkte)

Für beide Varianten gelten $Q_S = 3$ ms, $T_S = 10$ ms und anfangs drei Budgeteinheiten. Keine anderen Tasks, keine Zusatzkosten, keine Unterbrechungen außer Budgeterschöpfung, FIFO. A kommt bei 7 ms mit 3 ms Bedarf, B bei 10 ms mit 3 ms Bedarf. Bei gleichen Zeitpunkten werden Ankünfte und fällige Auffüllungen vor der Auswahl berücksichtigt.

Der **Deferrable Server** setzt bei 0, 10, 20, … sein Budget auf drei zurück; nicht verbrauchtes altes Budget wird dabei nicht addiert.

Für den **Sporadic Server** betrachten wir nur die hier eindeutigen aktiven Abschnitte ohne höher priorisierte Arbeit: Ein aktiver Abschnitt beginnt beim Serverstart t_a. Verbraucht er bis zum Leerwerden oder zur Budgeterschöpfung q, wird diese Menge bei $t_a + T_S$ zurückgegeben. Im Leerlauf bleibt ungenutztes Budget erhalten. Es gibt keinen pauschalen Reset bei 10 ms. Überlappende Auffüllungen während eines aktiven Abschnitts treten in dieser Aufgabe nicht auf.

1. Geben Sie beide Ausführungsintervalle des Deferrable Servers an. (2 Punkte)
2. Geben Sie beim Sporadic Server die Intervalle, die beiden Auffülltermine mit Mengen und die Antwortzeit von B an. (4 Punkte)
3. Wie viel CPU-Zeit beansprucht der Deferrable Server in $[7, 13)$? Warum kann man aus Q_S/T_S allein keinen Fristnachweis für niedrigere Tasks ableiten? (2 Punkte)

Musterlösung:

1. Deferrable: A läuft $[7, 10)$ und verbraucht das Budget der ersten Periode. Bei 10 ms setzt die nächste Auffüllung das Budget wieder auf drei. B läuft sofort $[10, 13)$.
2. Sporadic: A läuft $[7, 10)$. Sein Abschnitt begann bei 7, deshalb werden drei Einheiten bei 17 ms wieder verfügbar. B wartet von 10 bis 17, läuft $[17, 20)$ und ist bei 20 fertig. Die nächste Rückgabe von drei Einheiten liegt bei $17 + 10 = 27$ ms. Für B gilt $R_B = 20 - 10 = 10$ ms. Der Rückgabetermin wird vom Abschnittsbeginn berechnet, nicht vom Abschnittsende; er liegt also nicht bei 20 bzw. 30 ms.
3. Der Deferrable Server beansprucht in diesen sechs Millisekunden sechs CPU-Millisekunden: zwei volle Budgets unmittelbar an einer Periodengrenze. Sein nominaler Anteil ist zwar $3/10 = 30\,\%$, daraus folgt aber nicht „höchstens drei CPU-Millisekunden in jedem beliebig gelegenen 10-ms-Fenster“. Ein Analyseverfahren muss das tatsächliche Auffüll- und Burstverhalten berücksichtigen. Die einfache periodische Interferenzformel wird nicht ungeprüft auf ihn übertragen.

Die eingeschränkte SS-Aufgabe zeigt das klassische Prinzip an vollständigen aktiven Abschnitten. Sie ist keine vollständige Implementierungsregel für beliebige Prioritätswechsel und überlappende Auffüllungen.

Aufgabe 4: Überlast und Grenzen einer Implementierung (8 Punkte)

Ein Server bietet $Q_S = 2$ ms je $T_S = 10$ ms. Jede Anfrage braucht genau 0,5 ms CPU-Zeit. Nehmen Sie für die Ratenrechnung an, dass jedes Budget vollständig ausgeführt werden kann und genügend Arbeit vorliegt. Kosten und Auslassungen fehlen zunächst.

1. Wie viele solche Anfragen passen in ein Budget, welche langfristige Bedienrate ergibt sich? (2 Punkte)
2. Es kommen langfristig 500 Anfragen/s. Bestimmen Sie den Belegungszuwachs. Ein **vereinfachtes stetiges Ratenmodell** beginnt mit 20 freien Queueplätzen. Wann sind diese rechnerisch gefüllt? Ignorieren Sie für diese Näherung die periodischen Bursts. (2 Punkte)
3. Warum schützt eine größere Queue die periodischen Tasks nicht automatisch vor zusätzlicher CPU-Last? Nennen Sie eine begründete Reaktion bei voller Queue. (2 Punkte)
4. Ein normaler FreeRTOS-Thread verarbeitet „höchstens zwei Nachrichten, dann `osDelay(10)`“. Laufzeiten der Nachrichten sind unbekannt. Ist damit ein 2-ms-CPU-Server mit 10-ms-Periode nachgewiesen? Begründen Sie. (2 Punkte)

Musterlösung: Vier Anfragen je Budget, langfristig $4/0{,}010 = 400$ Anfragen/s. Der Rückstau wächst um $500 - 400 = 100$ Anfragen/s. Im ausdrücklich stetigen Näherungsmodell werden 20 freie Plätze nach $20/100 = 0{,}2$ s verbraucht. Für den wirklichen burstweisen Ablauf sind außerdem Phase und Zwischenbelegung zu untersuchen.

Eine Queue begrenzt Speicherplatz, nicht die Rechenzeit eines höher priorisierten Workers. Ein größerer Puffer verschiebt bei Dauerüberlast nur die Vollbelegung. Eine zulässige Politik kann neue Anfragen ablehnen und mitzählen; eine andere darf ausdrücklich veraltete Arbeit ersetzen, wenn der Datenvertrag dies erlaubt. Beliebige Ereignisflut kann nicht ohne Verluste und mit fester Antwortfrist verarbeitet werden.

Zwei Nachrichten sind eine Anzahlgrenze. Ohne begrenzte eigene CPU-Kosten folgt daraus kein 2-ms-Budget. `osDelay(10)` wartet relativ zum Aufruf; die vorangegangene Arbeit kommt zum Aktivierungsabstand hinzu. Es fehlen außerdem eine spezifizierte Auffüllregel und deren Durchsetzung. FreeRTOS setzt ein Serververfahren nicht allein durch den Threadnamen um.

Aufgabe 5: Noch Arbeit, aber kein Budget (4 Punkte)

Eine Anfrage wartet im Server. Der Prozessor ist sonst frei, das Budget aber erschöpft.

1. Weshalb kann Leerlauf richtig sein? (1 Punkt)
2. Welche zusätzlichen Last- und Bedienannahmen brauchen Sie für eine garantierte Antwortzeit jeder Anfrage? (2 Punkte)
3. Unterscheiden Sie die Warteschlange im Modell von einem Thread, der bloß aktiv seine Zeit prüft. (1 Punkt)

Musterlösung: Die Budgetpause begrenzt die Interferenz des Servers. Eine zusätzliche Verarbeitung ohne geregelte Hintergrund-/Rückgewinnungsregel würde den analysierten Vertrag ändern. Für Anfragefristen braucht man unter anderem begrenzte Ankunftslast, Anfragekosten, Rückstau und gesicherte Bedienmöglichkeiten. Ein aktives Prüfen macht einen hoch priorisierten Thread nicht Blocked; in einer realen Implementierung muss budgetbedingtes Warten geeignet blockieren oder der Server aus der Auswahl genommen werden.

Aufgabe 6: Einen Polling Server als LED-Modell selbst simulieren (12 Punkte)

Programmieren in C11 oder neuer, etwa 25–35 Minuten. Verwenden Sie die bekannte **M4-only-FreeRTOS-Basis aus Übung 3** mit **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Nur LD1/PB0 und LD2/PE1 werden benötigt. Die Projektvorbereitung nennt die CubeMX-Schritte. Die Datenstruktur und Implementierungsregeln stehen direkt in dieser Aufgabe; Sie schreiben die Anwendungslogik selbst.

Ein virtueller Schritt stellt eine Modell-Millisekunde dar. Zur Anzeige führt eine gewöhnliche Simulationstask nur ungefähr alle **100 reale Kernelticks** einen Schritt aus. Diese Verlangsamung ist keine CPU-Budgetmessung und implementiert keinen Scheduling-Server im FreeRTOS-Kernel.

```
typedef struct {
    uint32_t t;
    uint32_t pending;
    uint32_t budget;
    uint32_t served;
} Modell;
```

1. Implementieren Sie `uint32_t ModellSchritt(Modell *m)` für einen gültigen Zeiger. Der Anfangszustand ist in allen Feldern null. Bei den virtuellen Zeiten **1, 3, 7 und 8** kommt jeweils genau ein Auftrag mit einer Einheit Arbeit an. Erhöhen Sie zuerst `pending` für eine Ankunft. Bei `t % 6U == 0U` ist Pollzeit: Setzen Sie `budget` auf zwei, falls Arbeit wartet, sonst auf null. Bei `pending > 0U && budget > 0U` wird genau eine Einheit ausgeführt: beide Werte um eins vermindern, `served` um eins erhöhen, Rückgabewert eins. Sonst Rückgabewert null. Wird die Arbeit leer, verfällt Restbudget. Erhöhen Sie danach `t` um eins. Es gibt im endlichen Versuch weder Zählerüberlauf noch mehr als vier Aufträge. (4 Punkte)
2. Legen Sie in CubeMX genau zwei CM4-Threads an: `StartHeartbeatTask` mit Normal-Priorität und 512 Byte Stack, `StartSimulationTask` mit BelowNormal-Priorität und 1024 Byte Stack. Heartbeat schaltet LD2 um und blockiert mit `osDelay(500)`. Die Simulation hält ein lokales `Modell m = {0};`, wiederholt die Schritte 0 bis 17 und beginnt dann mit einem neuen Nullzustand. Sie schreibt ausschließlich LD1: an bei Rückgabewert eins, sonst aus, danach `osDelay(100)`. Prüfen Sie beide Delay-Rückgaben und die vom Generator angelegten Handles. Rufen Sie bei Fehlern `Error_Handler()` auf und stellen Sie sicher, dass dieser Fehlerpfad nicht zur regulären Ausführung zurückkehrt. Erhalten Sie genau eine Kernelinitialisierung, die zwei generierten Threadanlagen und einen Kernelstart. (4 Punkte)
3. Prüfen Sie Ihre Funktion zuerst auf Papier: In welchen Schritten muss LD1 an sein? Welchen Wert hat `served` nach Schritt 17? Nur als Papiervariante: Die erste Ankunft liegt bei **0 statt 1**; alle anderen Angaben bleiben gleich. Welche Bedienungsschritte ändern sich durch den Restbudgetverfall? Bauen und laden Sie anschließend das ursprüngliche Modell nur auf CM4, beobachten Sie das Muster und die unabhängige Heartbeat-LED. Geben Sie den tatsächlichen Prüfstatus an. (4 Punkte)

Ergebnis: eigener endlicher Modellschritt und zwei laufende Threads. Modellzeit, reale Kernelticks und tatsächliche CPU-Zeit sind drei verschiedene Größen. M7 bleibt deaktiviert, Taster und zusätzliche Timer werden nicht verwendet.

Musterlösung zu Aufgabe 6

```c
#include "main.h"
#include "cmsis_os2.h"
#include <stddef.h>
#include <stdint.h>

static void Lab06_Stop(void)
{
    Error_Handler();
    for (;;) {
        /* Fehlerpfad bleibt auch bei rueckkehrendem Error_Handler hier. */
    }
}

uint32_t ModellSchritt(Modell *m)
{
    if (m->t == 1U || m->t == 3U || m->t == 7U || m->t == 8U) {
        ++m->pending;
    }
    if (m->t % 6U == 0U) {
        m->budget = m->pending > 0U ? 2U : 0U;
    }
    uint32_t ran = 0U;
    if (m->pending > 0U && m->budget > 0U) {
        --m->pending;
        --m->budget;
        ++m->served;
        ran = 1U;
    }
    if (m->pending == 0U) {
        m->budget = 0U;
    }
    ++m->t;
    return ran;
}

void StartHeartbeatTask(void *argument)
{
    (void)argument;
    for (;;) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
        if (osDelay(500) != osOK) {
            Lab06_Stop();
        }
    }
}

void StartSimulationTask(void *argument)
{
    (void)argument;
```

```
    Modell m = {0};
    for (;;) {
        if (m.t == 18U) {
            m = (Modell){0};
        }
        const uint32_t ran = ModellSchritt(&m);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0,
                          ran != 0U ? GPIO_PIN_SET : GPIO_PIN_RESET);
        if (osDelay(100) != osOK) {
            Lab06_Stop();
        }
    }
}
```

Der Typ `Modell` aus der Aufgabenstellung steht vor diesen Funktionen im gemeinsamen Benutzerbereich der CM4-Datei. Der Zeiger auf das lokale Modell bleibt während der gesamten Simulationstask gültig; nur diese Task verändert es. `Lab06_Stop` wird im selben erhaltenen Benutzerbereich definiert; die zusätzliche Endlosschleife verhindert eine Fortsetzung auch dann, wenn der projektspezifische `Error_Handler` zurückkehrt.

Unmittelbar nach den beiden bereits vom Generator erzeugten `osThreadNew`-Aufrufen und vor `osKernelStart()` wird die folgende Prüfung im erhaltenen Benutzerbereich ergänzt. `stddef.h` stellt `NULL` bereit. Die vorgeschlagenen Tasknamen ergeben üblicherweise diese Handles; maßgeblich sind die tatsächlich generierten Namen. Weitere Threadanlagen oder Kernelinitialisierungen werden nicht eingefügt.

```
if (HeartbeatHandle == NULL || SimulationHandle == NULL) {
    Error_Handler();
    for (;;) {
        /* Bei einem Anlagefehler den Kernel nicht starten. */
    }
}
```

Die Handleprüfung bleibt in der Datei mit der generierten Threadanlage; sie benötigt keinen Zugriff auf die lokale Hilfsfunktion `Lab06_Stop` aus einer anderen Datei. Die Modell- und LED-Funktionen bleiben in ihren vorgesehenen Benutzerbereichen.

Ausführungsschritte: **6, 7, 12 und 13**. Nach dem bearbeiteten Schritt 17 gilt `t=18`, `pending=0`, `budget=0`, `served=4`. Die Anfrage bei 7 wird vor dem Schritt eingereiht; im selben Schritt wird noch der ältere Auftrag von 3 bearbeitet. Die einzelne Auftragsidentität wird hier nicht gespeichert, weil alle Aufträge genau eine Einheit benötigen. Das Zählmodell dient dem Budgetverständnis und ersetzt keine Nutzdatenqueue.

Papiervariante mit erster Ankunft bei null: Bei null wird eine Einheit sofort bedient; die Queue ist danach leer, deshalb verfällt die zweite Budgeteinheit. Bei 6 wartet nur der Auftrag von 3. Nach seiner Bedienung wird die Queue erneut leer und Restbudget verfällt. Die Aufträge von 7 und 8 warten deshalb bis 12 und werden in den Schritten 12 und 13 bedient. Die Bedienungsschritte sind **0, 6, 12 und 13**; `served` ist nach Schritt 17 weiterhin vier. Restbudget darf weder für die Ankunft bei 3 noch für die bei 7 aufbewahrt werden.

Die LD1-Pulse zeigen die Modellentscheidung, nicht die echte CPU-Laufzeit eines FreeRTOS-Servers. LD2 soll trotz der Modellpausen weiterlaufen. Ohne ausgeführten Boardlauf wird nur die logische Prüfung als durchgeführt angegeben.